

Exerciții — o platformă de la zero, cu atelierul înainte

16 exerciții · două servicii, un client, infrastructura lor — toate prin agent · lecția 3

Terenul de lucru: un folder **nou și gol**, lângă `ms\` — de exemplu `C:\Projects\teste claudel\velo`. Zero legătură cu platforma ta: alte repo-uri, alt cluster, alt domeniu. Ți trebuie **Rancher Desktop cu Kubernetes pornit** (contextul `rancher-desktop` e deja în kubeconfig-ul tău), Docker Hub și `gh`. **Ce se schimbă față de fișele 1 și 2:** acolo platforma exista și **tu știai răspunsul înainte să întrebi**. Aici nu există nimic și nu știe nimeni răspunsul — nici tu, nici agentul. De aceea fiecare verificare din fișa asta se scrie **înainte** de lucrul pe care îl verifică: gardul înainte de primul fișier, harta înainte de teritoriu, testul înainte de cod, lanțul înainte de al doilea serviciu.

Platforma: `velo` — stații de biciclete, cu capacitate și număr de biciclete; o cursă pornește dintr-o stație și se închide într-una. Patru repo-uri publice, create de tine la exercițiul 5:

Repo	Ce e
<code>statii-api</code>	stațiile: capacitate, câte biciclete sunt acum, scoate una / pune una
<code>curse-api</code>	cursele: pornire (scoate o bicicletă din stația A), închidere (o pune în stația B)
<code>velo-web</code>	clientul: stațiile, cursele deschise, pornește / închide o cursă
<code>velo-gitops</code>	singurul repo citit de ArgoCD: infra, chart-uri, Application-uri

Domeniul e ales pentru un singur motiv: are o regulă verificabilă cu două `curl`-uri — **suma bicicletelor din stații + cursele deschise = 30, întotdeauna**. Fără ea, nicio poartă din fișă n-ar avea ce să prindă.

Cum se citește fișa. Fiecare exercițiu are sarcina, un **gata când** pe care îl bifezi cu ochii tăi, de obicei o **poartă** (strici ceva cu mâna, verifici că unealta se supără), un **indiciu** și o linie de **teorie**. Ordinea contează: **teoria se citește înainte** de exercițiu — e pagina din ghid sau exercițiul din fișele vechi pe care stă acesta; **indiciul se citește după prima încercare**, nu înainte. Dacă ai nevoie de el de la primul rând, nu e o problemă — dar notează-ți că ai avut, e harta gurilor tale. Materialele la care trimite: `Ghid = GHID-CLAUDE-CODE.pdf` · `Cuvinte = CUVINTE-CHEIE.pdf` · `F1 / F2 = fișele 1 și 2` · `platforma ta = fișierele din ms\`, pe care le cunoști deja. Înainte de exercițiul 1: recitește Ghid pag. 1–2 și Cuvinte pag. 2 — harta pieselor și cine apasă butonul la fiecare.

Partea I — Atelierul înainte de orice: nu există nicio linie scrisă

Gardul pe un folder gol

1

Creează folderul, intră în el, pornește `claudel`. Înainte de orice altceva, `/permissions: deny` pe `kubectll` `apply / delete / scale / patch`, `helm install / upgrade / uninstall`, `argocd app sync / delete`, `git push`, `docker push`, `gh repo delete`. Apoi cere-i:

Instaleaza ArgoCD in clusterul rancher-desktop.

Și imediat după:

Ce contexte are kubectll, care e activ si ce ruleaza in el?

Gata când: prima a fost refuzată cu motivul pe ecran, a doua a răspuns — iar contextul activ e rancher-desktop, nu nucserver. Dacă e nucserver, oprește-te aici: ai două clustere în același kubeconfig și agentul nu știe care e cel adevărat. Schimbă contextul cu mâna, apoi continuă.

Indiciu: fișierul e `velo\.claude\settings.json` și are aceeași formă ca exemplul din Ghid pag. 3 — îl copiezi și adaugi trei reguli care acolo lipseau, pentru că pe `ms\` nu publicai imagini din atelier:

```
"Bash(docker push:*)", "Bash(gh repo delete:*)", "Bash(helm uninstall:*)"
```

În `allow` pune ce vei folosi de o sută de ori: `kubect! get / describe / logs`, `git status / log / diff`, `docker build`, `curl`. Contextul îl vezi cu `kubect! config current-context` și îl schimbi cu `kubect! config use-context rancher-desktop`. Primul refuz trebuie să vină de la `deny`, nu de la bunul-simț al agentului — dacă a refuzat „pentru că nu e bine”, n-ai testat gardul, ai testat starea lui de spirit. Pune-l să-ți arate comanda exactă pe care a vrut s-o ruleze.

Teorie: Ghid pag. 3 — Piesa 1, permisiuni, inclusiv „Unde se oprește gardul” · Cuvinte pag. 2 — `allow / deny / ask` · F1 ex. 3 (refuzul) și ex. 10 (gardul ocolit printr-un script).

Harta înainte de teritoriu

2

Scrie `CLAUDE.md` cu mâna ta, înainte să existe vreun repo. Nu e un rezumat — e un **contract**: cele patru repo-uri și rolul fiecăruia · namespace-ul `velo` · **un singur repo GitOps** · convenția de tag (sha7, scris de ci-bot, cu ghilimele) · contextul `kubect!` permis, și regula că orice alt context înseamnă *oprește-te* · regula cu cele 30 de biciclete. Apoi sesiune nouă și:

```
Ce servicii are platforma, in ce namespace ruleaza, si cine scrie
tag-ul din values.yaml? Si cate repo-uri git sunt acum in folderul asta?
```

Gata când: la primele trei a răspuns corect din `CLAUDE.md`, cu discul gol — iar la a patra a spus **zero**, nu a inventat. Harta spune patru, podeaua spune zero, și agentul trebuie să le spună pe amândouă.

Indiciu: pornește de la `ms\CLAUDE.md` al tău și de la exemplul din Ghid pag. 4 — **aceeași formă, alt conținut**: un tabel cu repo-urile, trei rânduri de nume (namespace, domeniu, Docker Hub) și o secțiune `## Reguli`. Scheletul, în 12 rânduri:

```
# Platforma velo
Patru repo-uri, un cluster local (Rancher Desktop), ArgoCD.
| Folder | Ce e |
|---|---|
| `statii-api/` | serviciu: statii, biciclete, scoate/pune |
| `curse-api/` | serviciu: curse; cheama statii-api |
| `velo-web/` | client static, config citit la runtime |
| `velo-gitops/` | SINGURUL repo citit de ArgoCD |
Namespace: velo. Domeniu: *.velo.localhost. Imagini: ion21/.
Tag = sha7 al commit-ului, scris de ci-bot, intre ghilimele.
## Reguli
- Context kubect! permis: rancher-desktop. Orice altul: te opresti.
- Clusterul se citește, nu se modifica. Orice schimbare = commit.
```

Ce nu scrii aici: stack-ul. Îl decide exercițiul 3 și-l adaugi atunci. Un rând în `CLAUDE.md` se citește la fiecare pornire — deci fiecare rând costă, și fiecare rând nescris e o decizie pe care o ia agentul în locul tău.

Teorie: Ghid pag. 4 — Piesa 2, `CLAUDE.md`, cu „Când NU: nu e loc de documentație” · F1 ex. 4 (ce ține minte o conversație) și ex. 11 · F2 ex. 3–4 (harta citită de un străin).

Planul, în mod plan

Shift+Tab până la **plan**. Apoi:

Propune structura platformei velo conform CLAUDE.md: ce contine fiecare repo, stack-ul celor doua servicii (acelasi limbaj pentru amandoua), baza de date, si drumul unei imagini din CI pana in pod.
Nu scrie niciun fisier.

Citește planul până la capăt. Caută în el **un lucru pe care nu l-ai cerut și nu e în CLAUDE.md** — un Kafka, un Redis, un al doilea repo GitOps, un tag `latest`, un `namespace` în plus. E aproape sigur acolo.

Gata când: `git status` și `dir` arată același folder ca înainte, **și** ai găsit lucrul inventat. Pentru el ai două opțiuni, și decizi tu, nu agentul: intră în `CLAUDE.md` cu `#`, sau îi spui să-l scoată din plan. Dacă planul e greșit de la rădăcină, `Esc Esc` și reformulează cererea — nu cârpi planul.

Indiciu: în bara de jos scrie modul curent; în plan agentul citește și scrie planul, dar orice scriere de fișier îți cere ieșirea din plan — dacă ți-a cerut-o, n-a înțeles „nu scrie niciun fișier”. Lucrurile pe care le inventează de obicei vin din reflexele lui de platformă „serioasă”: o coadă de mesaje între servicii (nu-ți trebuie — `curse-api` cheamă `statii-api` sincron, și exact asta testezi la ex. 11), un cache, un `Secret` scris de mână pentru baza de date (ex. 8 explică de ce nu există), un `latest`. Stack-ul: alege ce știi să citești — Spring Boot e alegerea evidentă la tine; dacă agentul propune altceva, întreabă-l de ce, și apoi decide tu. Decizia o scrii imediat în `CLAUDE.md`, cu `#`.

Teorie: Ghid pag. 2 — „Cele trei moduri” și de ce planul e cel mai folosit · Cuvinte pag. 1 — `Shift+Tab`, `Esc Esc`, `#` · F1 ex. 5 (butonul de stop).

Regula domeniului, scrisă înainte de API

Adaugă regula în `CLAUDE.md` fără să deschizi fișierul:

```
# Invariant: suma bicicletelor din statii + cursele deschise = 30,
intotdeauna. Orice ruta care schimba numarul de biciclete il pastreaza.
```

Apoi, tot în mod plan:

Scrie `CONTRACT.md`: rutele HTTP ale celor doua servicii, ce primesc, ce intorc. Pentru fiecare ruta care schimba numarul de biciclete, scrie ce se intampla cand ar incalca invariantul: statie goala, statie plina, cursa inchisa de doua ori, statie inexistentă.

Gata când: fiecare rută care scrie are **cazul ei de refuz**, explicit. Dacă una n-are — de obicei închiderea de două ori a aceleiași curse — tocmai ai găsit primul bug al platformei, înainte de prima linie de cod. Completează contractul, nu codul.

Indiciu: contractul e un tabel, nu proză. Forma pe care o aștepți, cu un rând complet ca model:

```
| Ruta | Primește | Intoarce | Refuza cand |
|---|---|---|---|
| POST /statii/{id}/scoate | - | 200 statia | 409 daca biciclete=0 |
| POST /curse | statiePlecare | 201 cursa | 409 daca statii-api refuza |
| POST /curse/{id}/inchide | statieSosire | 200 cursa | 409 daca e inchisa |
```

Codul de răspuns pentru „refuz pe regulă de domeniu” e `409 Conflict`, nu `400` (cererea e corectă, starea nu permite) și nu `500`. Rutele care doar citesc (`GET /statii`, `GET /curse?deschise=true`) n-au coloana de refuz, dar sunt cele pe care le adună `invariant.sh` la ex. 11 — scrie-le acum, cu forma exactă a răspunsului, ca să ai ce aduna.

Teorie: Cuvinte pag. 1 — # adaugă o linie în `CLAUDE.md` · pe platforma ta nu există un fișier de contract între `importer-service` și `data-service` — topicul Kafka e tot contractul; aici îl scrii explicit tocmai ca să ai cu ce compara codul la ex. 14.

Granița mâinii tale. Regula cursului nu se schimbă: *agentul nu atinge clusterul; agentul schimbă repo-ul; clusterul se schimbă singur*. Tot ce faci tu cu mâna în fișa asta este lista de mai jos, și **numai** ea: `gh repo create` de patru ori · secretele de CI în GitHub, la Settings → Secrets · trei linii în `hosts` · `helm install` pentru ArgoCD · `kubectl apply` pe `bootstrap/root.yaml`. Două `apply`-uri în toată fișa. Exercițiul 16 le numără.

Partea II — Scheletul: un singur serviciu, cap-coadă, înainte de al doilea

Testul înainte de cod

5

Creează cele patru repo-uri pe GitHub, publice, și clonează-le în `velo\`. Apoi, în `statii-api`:

Scrie DOAR testele pentru `statii-api`, după `CONTRACT.md`: scoaterea dintr-o stație cu 0 biciclete e refuzată, punerea într-o stație plină e refuzată, și totalul nu se schimbă după orice succesiune de operații. Nicio implementare. Rulează-le: trebuie să cadă.

Gata când: testele cad — și cad **din motivul corect**. Citește mesajul fiecăruia: „implementarea lipsește / întoarce nimic” e corect; o eroare de sintaxă în test sau o dependență lipsă nu e un test care cade, e un test care nu rulează.

Indiciu: repo-urile, dintr-o linie fiecare, din `velo\`:

```
gh repo create nimigeanconstantinon/statii-api --public --clone
```

Într-un limbaj compilat „testul cade” are nevoie de un schelet care compilează: clasa controller-ului cu metodele goale (sau care aruncă `NotImplemented`). Cere-i explicit *scheletul minim ca testele să compileze și să cadă pe aserțiune*. Trei teste, numite după rândurile din contract — `scoateDinStatieGoala_409`, `puneInStatiePlina_409`, `totalulRamane30_dupa200operatii`. Al treilea e cel care contează: celelalte două verifică o linie, al treilea verifică regula.

Teorie: Ghid pag. 11 — „De unde știu că unealta asta poate să-mi spună nu” · F1 ex. 8 — un test care trece pe cod stricat nu e un test · pe platforma ta: testele cu `Testcontainers` din `data-service` rulează deja în CI — aceeași mecanică, alt obiect.

Implementarea, și poarta pe test

6

Implementează `statii-api` până trec toate testele. Nu modifică testele. Datele initiale: 3 stații, 30 de biciclete în total.

Rulează testele din sesiune, cu `!`, ca ieșirea să intre în conversație. Când sunt verzi, **poarta**: deschide tu codul, scoate cu mâna verificarea „stație goală”, rulează din nou.

Gata când: verde pe cod bun, roșu pe cod stricat, apoi `git diff` gol. Dacă pe cod stricat a rămas verde, nu codul era problema — testul era un afiș. Pune-l să-l repare pe test, nu pe cod. La final: `/init`, și compară ce a scris cu harta ta. Ce a scris și **nu poți verifica din fișiere** se taie.

Indiciu: `!mvn test` (sau echivalentul stack-ului tău) — semnul `!` rulează comanda în shell și pune ieșirea în conversație, deci agentul vede exact ce vezi tu. Sabotajul: caută linia cu `if (biciclete == 0)` (sau cum a numit-o) și comentează-o; dacă testul `scoateDinStatieGoala_409` rămâne verde, înseamnă că testul nu cheamă de fapt ruta, sau verifică doar că „nu a aruncat”. `/init` scrie un `CLAUDE.md` în `statii-`

`api\` — e al doilea, pe lângă cel din `velo\`, și se citesc amândouă când pornești din `velo\`. Păstrează din el doar ce e verificabil: comanda de build, comanda de test, portul, numele profilului. Propozițiile de genul „serviciul e proiectat să fie scalabil” se taie — nu le poate verifica nimeni.

Teorie: Cuvinte pag. 1 — `!` și `/init` · Ghid pag. 4 — „fiecare rând din `CLAUDE.md` se citește la fiecare pornire” · F1 ex. 8 (poarta).

Dockerfile și CI, cu două scanări înăuntru de la prima rulare

7

Scrie Dockerfile multi-stage și `.github/workflows/ci.yml` pentru statii-api: teste, gitleaks pe repo, build imagine, trivy pe imagine (CRITICAL opreste pipeline-ul), push pe Docker Hub cu tag sha7, apoi bump-ul tag-ului în `velo-gitops`, la `business/statii-api/values.yaml`, facut de ci-bot, cu tag-ul între ghilimele. Secretele de CI nu le scrii tu nicaieri: le pun eu în GitHub.

Pune cu mâna, în Settings → Secrets, tokenul de Docker Hub și un token cu drept de scriere **doar** pe `velo-gitops`. Scrie în `CLAUDE.md` data la care expiră. Push. Apoi **două porți**, pe un branch separat:

- un commit cu o linie `API_KEY=abc123def456` într-un fișier oarecare;
- un commit care schimbă base image-ul cu unul de acum doi ani.

Gata când: run-ul pe `main` e verde și imaginea e pe Docker Hub; run-ul cu secretul e **roșu pe gitleaks**; run-ul cu imaginea veche e **roșu pe trivy**. Trei run-uri, trei culori pe care le poți arăta. Apoi branch-ul se șterge.

Indiciu: pipeline-ul tău din `data-api/.github/workflows/ci.yml` e modelul — are deja build, teste, push și bump. Îi lipsesc două job-uri și are o linie de schimbat. Job-urile, ca pași de GitHub Actions:

```
- uses: gitleaks/gitleaks-action@v2
- uses: aquasecurity/trivy-action@master
  with: { image-ref: "ion21/statii-api:${{ env.TAG }}",
        severity: CRITICAL, exit-code: "1" }
```

Linia de schimbat e bump-ul. Al tău face `sed -i -E "s|^(\ tag: ).*|\1${tag}|"` — scrie valoarea fără ghilimele, deci primul sha7 numai din cifre devine număr în YAML. Variante care pun ghilimelele **întotdeauna**, nu doar când îți amintești:

```
yq -i '.image.tag = strenv(TAG)' business/statii-api/values.yaml
```

yq păstrează tipul *string* și pune singur ghilimelele când valoarea ar arăta ca un număr — regula e în unealtă, nu în memoria ta. Tokenul pentru `velo-gitops`: un *fine-grained PAT* cu `Contents: write` pe un singur repo; data de expirare în `CLAUDE.md` pentru că un PAT expirat nu dă eroare — bump-ul pur și simplu nu mai apare. Base image vechi pentru poarta Trivy: `eclipse-temurin:17.0.1_12-jre` (2021) are CRITICAL-uri sigur. gitleaks prinde `API_KEY=abc123def456`? Valoarea e scurtă — dacă nu o prinde, pune una de 32 de caractere hexa: regulile lui au praguri de entropie, și e bine să afli acum, pe o linie falsă, unde e pragul.

Teorie: F2 ex. 8 — „cine scrie valoarea de obicei”, proba pe copie · Cuvinte pag. 2 — *bump*, *drift* · programa DevOps: lecția 13 (imagini sigure, Trivy) și lecția 19 (secret scanning) — cele două marcate *de făcut* la tine · platforma ta: `ci.yml:51`, linia cu `sed`.

velo-gitops: ArgoCD instalat de tine, o singură dată — restul vine din repo

8

Instalează ArgoCD cu mâna, cu `helm install`, în `rancher-desktop`. E primul dintre cele două `apply`-uri ale fișei, chiar dacă se cheamă altfel. Apoi:

Scrive `velo-gitops: bootstrap/root.yaml` (App-of-Apps), `argo-apps/` cu un Application pentru `infra` și unul pentru `statii-api`, `infra/` cu operatorul `cloudnative-pg` și un Cluster Postgres cu o instanță în namespace-ul `velo`, chart-ul `statii-api` cu values (image, tag între ghilimele, datele de conectare citite din secretul generat de operator, ingress pe `statii.velo.localhost`). Nu rula nimic în cluster.

Al doilea `apply`: `kubectl apply -f bootstrap/root.yaml`. Trei linii în `hosts` pentru `statii / curse / app.velo.localhost`. Apoi aștepti. Sesiunea asta e cea mai lungă din fișă: când `/context` trece de jumătate, `/compact` înainte să continui.

Gata când: `kubectl get applications -n argocd` arată `root` și copiii **Synced / Healthy**, pod-ul `statii-api` e Running, iar `curl http://statii.velo.localhost/statii` întoarce cele 3 stații cu 30 de biciclete. Nicio comandă în afara celor două nu a atins clusterul — dacă a fost nevoie de a treia, notează-o: e o gaură în repo și o reparați acum, nu la exercițiul 16.

Indiciu: structura e cea din `argo-ms-gitops` al tău, redusă la ce ai nevoie:

```
velo-gitops/  
  bootstrap/root.yaml      Application -> argo-apps/  
  argo-apps/  
    cnpg-operator.yaml     chart-ul oficial, wave -1  
    velo-db.yaml           infra/velo-db/, wave 0  
    statii-api.yaml        charts/statii-api/, wave 1  
  infra/velo-db/cluster.yaml Cluster, instances: 1  
  charts/statii-api/       chart-ul tau de microserviciu
```

Ordinea e totul, și o dau sync wave-urile: operatorul instalează CRD-ul `Cluster`; până nu există CRD-ul, manifestul din `infra/velo-db/` nu poate fi nici măcar validat — de aici valul `-1` pe operator și `0` pe baza de date, exact ca la tine între `cloudnative-pg` și clusterul lui. Secretul: pentru un `Cluster` numit `velo-db` operatorul creează singur `velo-db-app`, cu cheile `username`, `password`, `dbname`, `host` — în values trimiți doar `secretKeyRef` spre el. Ingress-ul: Rancher Desktop vine cu Traefik, deci `ingressClassName: traefik`; în `hosts`: `127.0.0.1 statii.velo.localhost curse.velo.localhost app.velo.localhost`. Instalarea ArgoCD, cele două comenzi ale tale:

```
helm repo add argo https://argoproj.github.io/argo-helm  
helm install argocd argo/argo-cd -n argocd --create-namespace
```

Dacă `statii-api` e Synced dar nu Healthy, uită-te la pod înainte de orice: aproape sigur e conectarea la bază — numele secretului sau al cheii.

Teorie: platforma ta — `argo-ms-gitops/bootstrap/root.yaml`, `argo-apps/`, `infra/cloudnative-pg/README.md` · programa DevOps: lecția 32 (operatori) și 34 (App-of-Apps) — amândouă *gata* la tine · Cuvinte pag. 1 — `/context`, `/compact` · Ghid pag. 10, depanare — „răspunsurile ratează detalii”.

Lanțul, cap-coadă, pe un singur serviciu

9

Schimbă un lucru vizibil în `statii-api` — numele unei stații — commit, push. Apoi **nu faci nimic**, doar te uiți, cu ceasul lângă tine, la fiecare verigă:

1. run-ul din GitHub Actions — ora la care s-a terminat;
2. tag-ul nou pe Docker Hub;
3. commit-ul lui ci-bot în `velo-gitops`, cu ghilimele în jurul tag-ului;
4. Application-ul: OutOfSync → Synced;
5. pod-ul nou, cu tag-ul nou în `kubectl describe`;
6. `curl` întoarce numele schimbat.

Gata când: ai cele șase ore notate, una sub alta, și `git log business/statii-api/values.yaml` în `velo-gitops` are **un singur autor** pentru bump: ci-bot. Niciun tag scris de mâna ta, niciodată — altfel lanțul are deja o verigă pe care o ții tu.

Indiciu: câte o comandă pentru fiecare verigă, ca să nu te uiți în cinci tab-uri:

```
gh run watch # 1
git -C velo-gitops log -1 --format='%an %ci' \
  -- business/statii-api/values.yaml # 3
kubectl get app -n argocd -w # 4
kubectl get pod -n velo -o jsonpath='{..image}' # 5
```

Veriga cea mai lentă e 3 → 4: ArgoCD **nu e anunțat** de commit, ci întreabă repo-ul din trei în trei minute. Dacă fereastra ta între 3 și 4 e de 0–3 minute, e normal; dacă e mai mare, nu e ArgoCD, e altceva. Pentru git log-ul final: dacă apare numele tău ca autor pe `values.yaml`, întreabă-te la ce exercițiu ai scris acolo cu mâna — probabil la 8, când ai pus prima valoare. Prima valoare e singura excepție legitimă; de la a doua încolo, ci-bot.

Teorie: Cuvinte pag. 2 — „Vocabularul platformei tale”: lanțul, bump, drift · F1 ex. 7 (prima unealtă) · F2 ex. 11 (ordinea și fereastra dintre schimbări, în minute).

Partea III — Al doilea serviciu și clientul: aceleași reguli, fără mâna ta

Comanda /serviciu-nou

10

Înainte să scrii o linie din `curse-api`:

Fa o comandă `/serviciu-nou <nume>` care produce pentru un serviciu nou tot ce are `statii-api` în afara codului de domeniu: `Dockerfile`, `pipeline-ul` cu ambele scanări și bump-ul, `chart-ul` și `Application-ul` în `velo-gitops`, `ingress-ul` pe `<nume>.velo.localhost`, randul din `CLAUDE.md`. Pleacă de la ce există în `statii-api`, nu inventa.

Rulează `/serviciu-nou curse-api`. Deocamdată serviciul e un hello-world; nu contează. Apoi **poarta**, care decide arhitectura în locul tău: schimbă un pas de scanare — pragul lui `trivy`, de exemplu — **într-un singur loc**, și rulează `pipeline-ul` ambelor servicii.

Gata când: `Application-ul curse-api` e Synced cu hello-world-ul înăuntru, **și** ambele servicii au primit schimbarea de scanare dintr-o singură modificare. Dacă a trebuit să modifice în două locuri, comanda a copiat `pipeline-ul` — și ai acum două fișiere care vor diverge. Pune-l să refacă: un workflow comun, chemat de fiecare serviciu.

Indiciu: comanda e `velo\.claude\commands\serviciu-nou.md`, cu aceeași formă ca `/lant` din Ghid pag. 5: `description`, `argument-hint: [nume]`, iar în text `$ARGUMENTS` e numele serviciului. Poarta trece doar cu un **reusable workflow**: `pipeline-ul` întreg stă o singură dată, în `velo-gitops/.github/workflows/ci.yml`, cu `on: workflow_call` și un `input` pentru numele serviciului; fiecare serviciu are un `ci.yml` de șase rânduri:

```
on: { push: { branches: [main] } }
jobs:
  ci:
    uses: nimigeanconstantin/velo-gitops/.github/workflows/ci.yml@main
    with: { service: curse-api }
    secrets: inherit
```

Fiecare repo de serviciu are nevoie de aceleași două secrete — le pui o dată pe organizație... nu ai organizație, deci de două ori, cu mâna. Notează-o ca a treia comandă manuală pe serviciu, și întreabă-te la ex.

16 dacă e în lista de la începutul fișei. Nu e. Asta e o gaură cunoscută, și e în regulă să fie — dar scrisă.

Teorie: Ghid pag. 5 — Piesa 3, comanda: `$ARGUMENTS`, `allowed-tools`, `description` · F2 ex. 7 — cele trei workflow-uri comparate linie cu linie; aici diferența nu mai are unde să apară · GitHub Docs: „Reusing workflows”.

11

curse-api, și regula care trece acum prin două servicii

Ca la exercițiile 5–6, testele întâi, apoi:

Implementează curse-api după `CONTRACT.md`: pornirea unei curse cheamă statii-api să scoată o bicicletă din stația de plecare, închiderea o pune în stația de sosire. Dacă statii-api refuză, cursa nu se creează. Testele rămân neschimbate.

Când totul e verde și deployat, **poarta** — un script, nu o părere:

Scrie `invariant.sh`: face 50 de operații la întâmplare pe API-ul din cluster — porniri, închideri, închideri repetate, statii inexistente, statii goale — apoi adună bicicletele din statii cu cursele deschise și afișează suma. Trebuie să fie 30.

Gata când: scriptul dă 30 de trei ori la rând. Apoi strici cu mâna: închiderea cursei nu mai cheamă pune. Scriptul trebuie să dea mai puțin de 30. Readu linia, `git diff gol`.

E cel mai greu exercițiu din fișă, și e normal să stea câteva zile. O bicicletă scoasă din stație de un serviciu și o cursă nesalvată de celălalt înseamnă o bicicletă dispărută: suma dă 29 și nimeni nu știe de ce. Agentul va spune „gata” înainte ca scriptul să spună 30. Scriptul are dreptate. Pune-l pe agent să caute până când cele două sunt de acord.

Indiciu: cele trei locuri unde dispare bicicleta, în ordinea în care le vei întâlni: (1) scoate a reușit, salvarea cursei a picat → bicicleta e nicăieri; fixul e **compensarea**: dacă salvarea pică, chemi pune înapoi pe aceeași stație. (2) închiderea de două ori: a doua cheamă iar pune → suma dă 31; fixul e să verifici starea cursei **înainte** de apel, nu după. (3) scoate a reușit, cursa s-a salvat, dar răspunsul HTTP s-a pierdut și clientul a reîncercat → două curse, o bicicletă; asta n-o rezolvi acum — o scrii în `CONTRACT.md` ca limită cunoscută, și ai aflat ce înseamnă „idempotent”. Scriptul, miezul lui:

```
for i in $(seq 50); do
  case $((RANDOM % 4)) in
    0) curl -s -o /dev/null -X POST $CURSE/course -d '{"statie":1}' ;;
    1) curl -s -o /dev/null -X POST $CURSE/course/$((RANDOM%20))/inchide \
      -d '{"statie":2}' ;;
    2) curl -s -o /dev/null -X POST $CURSE/course -d '{"statie":99}' ;;
    3) curl -s -o /dev/null -X POST $STATII/statii/1/scoate ;;
  esac
done
s=$(curl -s $STATII/statii | jq '[.[]].biciclete' | add)
c=$(curl -s "$CURSE/course?deschise=true" | jq 'length')
echo "statii=$s curse=$c suma=$((s+c))"
```

Cazul 3 e deliberat: cheamă scoate direct, fără cursă — tocmai ca să vezi că statii-api singur nu poate ține invariantul; îl ține doar perechea. Dacă suma iese constant sub 30 cu exact numărul cazurilor 3, ai înțeles de ce ruta scoate nu ar trebui să fie publică — și ai o decizie de contract de luat.

Teorie: F1 ex. 8 și Ghid pag. 11 — poarta · lecția 42 din programa DevOps (aplicația observabilă): când suma dă 29, log-urile celor două servicii sunt singurul loc unde vezi care apel a reușit și care nu · cuvintele de căutat, pentru după: *compensare*, *idempotență*.

Fa un skill /invariant care ruleaza invariant.sh pe mediul local si raporteaza suma. Descrierea lui trebuie sa-l faca sa porneasca singur ori de cate ori se schimba o ruta care muta biciclete, fara sa-l chem eu.

Poarta pe descriere — într-o sesiune nouă, fără să pomenști skill-ul:

Adauga in statii-api o ruta care muta 5 biciclete dintr-o statie in alta.

Gata când: după ce a scris ruta, a rulat verificarea **singur** și ți-a raportat suma. Și ai citit ce a găsit: transferul într-o stație aproape plină trece de 30 pe o stație și sub 0 pe cealaltă? Dacă skill-ul nu a pornit, nu codul e de vină — descrierea lui nu s-a potrivit cererii. Rescrie descrierea, nu cererea.

Indiciu: skill-ul e un folder: `velo\claude\skills\invariant\` cu `SKILL.md` și, lângă el, `invariant.sh` — scriptul stă în skill, nu undeva în repo-uri, ca să-l poată rula din orice context. Descrierea e tot exercițiul. Compară:

```
description: Verifica invariantul platformei velo. # nu porneste
description: Ruleaza verificarea celor 30 de biciclete. Se
foloseste DUPA orice modificare a unei rute care scoate, pune
sau muta biciclete (statii-api sau curse-api), inainte sa
raportez ca am terminat. # porneste
```

Prima spune *ce face*; a doua spune *când* — cu cuvintele pe care le folosești tu când ceri („rută”, „mută biciclete”). Agentul alege skill-ul comparând descrierea cu cererea ta, nu cu intenția ta. Dacă nici a doua nu pornește, Ghid pag. 10 are rândul exact din tabelul de depanare.

Teorie: Ghid pag. 6 — Piesa 4, skill-ul, și „nu se testează cerându-i-l pe nume” · Ghid pag. 10 — regula de alegere comandă / skill / hook și tabelul de depanare · F1 ex. 13, F2 ex. 9 și 14.

Clientul, pe contractul real

Scrie velo-web: pagina cu statiile si cursele deschise, pornire si inchidere de cursa. URL-urile celor doua API-uri se citesc la runtime dintr-un config/config.json servit langa pagina, NU din variabile de build. Rutele le iei din CONTRACT.md, nu le inventezi.

Apoi `/serviciu-nou velo-web`, și clientul intră în lanț ca celelalte două. **Poarta:** oprește `statii-api` — dar **prin repo:** `replicas: 0` în values, commit, push, aștepți ArgoCD.

Gata când: cu backendul oprit, clientul arată o **eroare**, nu datele de dinainte și nu date inventate. Apoi `replicas: 1`, același drum, datele revin. Dacă prima ta pornire a fost să scrii `kubectl scale`, întoarce-te la exercițiul 1: gardul te-ar fi oprit, dar regula e pentru tine, nu doar pentru agent.

Indiciu: ai făcut deja jumătatea grea pe `client-microserv-vite`: Dockerfile-ul multi-stage `node` → `nginx` e modelul, și știi deja de ce `VITE_*` nu merge — valorile intră în bundle la `npm run build`, deci aceeași imagine nu poate merge în două medii. Aici `config.json` nu e în imagine deloc: e un `ConfigMap` din values, montat în `nginx` la `/usr/share/nginx/html/config/`, și pagina îl citește cu un `fetch` la pornire. Pentru poartă: dacă `fetch` spre `statii-api` pică, pagina trebuie să **arate** că a picat — nu să țină lista veche în memorie și nu să pornească de la un array gol care arată ca „nicio stație”. „Nicio stație” și „nu pot citi stațiile” sunt două stări diferite și clientul trebuie să le afișeze diferit. Al treilea rând în `hosts` e pentru `app.velo.localhost`; CORS nu apare dacă `velo-web` cheamă API-urile prin aceleași host-uri `*.velo.localhost` din browser — dar **apare** dacă ai pus în `config.json` numele serviciilor din cluster. Config-ul e pentru browser, nu pentru pod.

Teorie: platforma ta — Dockerfile-ul din `client-microserv-vite` și `public/config/config.json` (aceeași idee, aici fără nimic sensibil în el) · programa DevOps: lecția 11, „config de runtime în imagine” — *parțial* la tine · F1 ex. 3 — refuzul la `scale` ; aici vezi drumul corect pentru aceeași nevoie.

Partea IV — Harta, codul și clusterul: cine păzește ce

Subagentul harta-vs-cod

14

Fa un subagent "harta-vs-cod". Citeste CLAUDE.md, CONTRACT.md si cele patru repo-uri. Intoarce trei liste: (a) ce spune harta si codul nu face, (b) ce face codul si harta nu spune, (c) ce afirma harta si nu se poate verifica din niciun fisier. Nu propune fix-uri.

Gata când: lista (b) **nu e goală** — ruta de transfer de la exercițiul 12 nu e în `CONTRACT.md` , iar expirarea tokenului de la exercițiul 7 e un candidat bun pentru (c). Pentru fiecare rând decizi: harta se actualizează sau codul se scoate. „Amândouă sunt în regulă” nu e o decizie.

De ce al doilea rol: agentul care a scris ruta e ultimul care observă că n-a trecut-o în contract. Subagentul nu e mai deștept — doar nu a scris-o el.

Indiciu: fișierul e `velo\.claude\agents\harta-vs-cod.md` , cu forma lui `sanatate-cluster` din Ghid pag. 7 — `name` , `description` , `tools` — doar că aici nici nu are nevoie de `kubectl: tools: Read, Grep, Glob` și atât. Dă-i metoda, nu doar scopul: *pentru fiecare rând din CONTRACT.md caută ruta în cod; apoi pentru fiecare rută găsită în cod (@PostMapping , @GetMapping ...) caut-o în CONTRACT.md* — cele două direcții dau listele (a) și (b). Lista (c) e cea pe care o va umple prost la început: tot ce e afirmație despre lume, nu despre fișiere („tokenul expiră pe...” , „clusterul e local”) intră acolo. Îl chemi cu `Foloseste harta-vs-cod pe toata platforma`. Dacă întreabă lucruri pe care le știai — e normal, pornește fără conversația ta; pune-i în cerere ce-i trebuie.

Teorie: Ghid pag. 7 — Piesa 5, subagentul, contextul separat · F2 ex. 10 (recenzentul care citește doar diff-ul) și ex. 12 (de ce subagent și nu aceeași conversație) · Cuvinte pag. 2 — „context separat”.

Hook-ul care ține harta sinceră

15

Un singur criteriu, altul decât cele din fișele 1 și 2:

Pune un hook înainte de git commit: daca diff-ul adauga o ruta HTTP noua intr-un serviciu, sau un Application nou in velo-gitops, si nici `CONTRACT.md` nici `CLAUDE.md` nu sunt in acelasi commit, opreste commit-ul cu exit 2 si spune ce lipseste.

Gata când: ai văzut amândouă rezultatele: rută nouă fără contract → oprit, cu motivul; aceeași rută plus o linie în `CONTRACT.md` → trece. Două teste, nu unul. De aici încolo lista (b) a subagentului ar trebui să rămână goală — verific-o peste o săptămână.

Indiciu: `pre-commit.sh` din Ghid pag. 8 e scheletul: primește JSON-ul pe intrare, se uită dacă e un `git commit`, altfel `exit 0` imediat. Miezul nou, după acel filtru:

```
adaug=$(git diff --cached -U0 | grep -E '^+' | \
  grep -cE '@(Post|Get|Put|Delete)Mapping|argo-apps/.*\yaml')
harta=$(git diff --cached --name-only | grep -cE 'CONTRACT.md|CLAUDE.md')
if [ "$adaug" -gt 0 ] && [ "$harta" -eq 0 ]; then
  echo "BLOCAT: ruta/Application nou fara CONTRACT.md sau CLAUDE.md." >&2
  exit 2
fi
```

Două lucruri de verificat înainte să dai vina pe script: fișierul e executabil și `jq` există în shell-ul din care rulează Claude (pe Windows, Git Bash îl are doar dacă l-ai pus). Și testul negativ e la fel de important ca cel pozitiv: un hook care blochează și commit-urile fără rute noi e mai rău decât niciun hook — îl vei dezactiva într-o săptămână. Pentru `argo-apps`: hook-ul rulează în repo-ul în care se face commit-ul, deci criteriul pe Application se declanșează doar când comiți în `velo-gitops` — e corect așa.

Teorie: Ghid pag. 8–9 — Piesa 6, hook-ul: `PreToolUse`, `exit 2`, „se testează stricând, nu citind”, „Când NU: un hook prost e mai rău decât niciun hook” · Ghid pag. 10 — „hook-ul nu rulează” · F1 ex. 15, F2 ex. 13.

16

Distruge clusterul. Reconstruiește-l din repo.

În Rancher Desktop: *Reset Kubernetes*. Clusterul dispare cu tot ce era în el. Apoi **exact cele două comenzi** de la exercițiul 8 — `helm install` pentru ArgoCD și `apply` pe `root.yaml` — și nimic altceva. Verificarea finală nu e o sesiune, e o comandă într-un script:

```
claude -p "Verifica: toate Application-urile Synced si Healthy, cele trei
ingress-uri raspund, invariant.sh da 30. Raspunde OK sau lista problemelor."
```

Gata când: răspunsul e OK și **numărul de comenzi date de mâna ta a fost doi**. Dacă a fost nevoie de a treia — un namespace, un secret, un `hosts` uitat — aia e gaura din repo: o scrii în repo, distrugi din nou, numeri din nou.

Indiciu: *Reset Kubernetes* e în Rancher Desktop la Troubleshooting; păstrează imaginile din cache, deci reconstruirea e mai rapidă decât prima instalare. Ce va pica la prima reconstruire, în ordinea probabilității: (1) `velo-db` `OutOfSync` pentru că CRD-ul operatorului nu era încă instalat când ArgoCD a încercat `Cluster`-ul — dacă ai pus wave-urile la ex. 8, nu; dacă nu, acum le pui; (2) namespace-ul `velo` — cine îl creează? Dacă l-ai creat tu cu mâna la ex. 8 fără să notezi, acum afli: pune `CreateNamespace=true` în `syncOptions`; (3) datele — baza e nouă, deci cele 30 de biciclete trebuie să vină din seed-ul aplicației, nu din ce ai pus tu cu `curl` cândva. `claude -p` rulează fără sesiune și fără să te întrebe — pornește-l din `velo\`, ca să vadă `CLAUDE.md` și gardul, și pune-l într-un `verifica.sh` ca să-l rulezi de fiecare dată când ți se pare că totul merge.

Teorie: Cuvinte pag. 1 — `claude -p`, „răspunde și iese” · Ghid pag. 1 — regula atelierului · F2 ex. 16 (jurnalul de o zi: ce a rămas neterminat) · programa DevOps: lecția 25, unde la tine scrie „clusterul e făcut manual” — aici clusterul rămâne manual pe bună dreptate, și tot restul nu.

Ce au în comun toate cele 16: nicio verificare din fișă nu a fost scrisă după lucrul pe care îl verifică. Gardul înainte de fișier, harta înainte de repo, testul înainte de cod, lanțul înainte de al doilea serviciu, scriptul înainte de „gata”. Agentul a scris aproape tot codul, și tot codul a trecut printr-o poartă pe care ai putut-o vedea cu ochii tăi: un run colorat, un pod, un `curl`, o sumă. Partea grea n-a fost codul. A fost să decizi, de fiecare dată, ce trebuie să existe înainte.